

Experiment 03 · SPI – Four-Wire Full-Duplex Link

SPI2 master and slave exchanging a four-byte frame under one chip-select

NUCLEO-F446RE
× 2
STM32F446RET6,
Cortex-M4

SPI2 (APB1)
peripheral used

250 kHz, mode 0
line settings

**5 (NSS, SCK,
MOSI, MISO, GND)**
wires between boards

Bare metal
CMSIS registers, no
HAL, no CubeMX

1 Aim and specification

Two NUCLEO-F446RE boards are made to talk to each other over **SPI** only. The board programmed as **master** reads its blue B1 user button and measures how long it is held; the board programmed as **slave** has no button and only reacts to what arrives on the SPI link, blinking its green LD2 LED in one of two clearly distinguishable patterns. After programming, both boards run from any 5 V USB supply with no PC attached.

1.1 Behavioural specification

Stimulus on the master	Measured press	Command transmitted	Slave LD2 pattern	LED-to-LED spacing	Total
Short click	< 1000 ms	0xA1 CMD_SHORT	3 blinks · 150 ms ON / 150 ms OFF	300 ms	0.9 s
Press and hold	≥ 1000 ms	0xA2 CMD_LONG	3 blinks · 1000 ms ON / 1000 ms OFF	2000 ms	6.0 s

The long-press pattern is the point of the experiment: 1000 ms on plus 1000 ms off means the slave LED starts a blink every **2.0 s**, against 300 ms for a short click. Both numbers are single `#define`s at the top of the slave listing (`LONG_ON_MS` , `LONG_OFF_MS` , `LONG_REPEAT`), so the timing can be retuned without touching the protocol code.

1.2 Status indication on the master

Indication	When	Meaning
Blue	steady while the frame is on the wire	transmission in progress
Green + one 250 ms LD2 flash	after a valid acknowledgement	the slave received the command
Red + three fast LD2 flashes	acknowledgement missing or wrong	wiring, power or protocol fault

The RGB LED is optional - set `USE_RGB` to 0 in the master listing and the on-board green LD2 alone carries the same information. If your RGB LED is common-anode, set `RGB_COMMON_CATHODE` to 0 instead of rewiring.

1.3 Frame format

All four experiments in this series use the same three-byte payload so that the protocols can be compared fairly:

Byte	Value	Purpose
1	0xAA	sync pattern – alternating bits, easy to recognise on an oscilloscope and unlikely to appear in noise
2	0xA1 or 0xA2	the command itself
3	~cmd (0x5E / 0x5D)	one's-complement checksum; the receiver acts only if it matches

2 Apparatus

Qty	Item	Notes
2	NUCLEO-F446RE development board	one programmed as master, one as slave
1 set	Male-male jumper wires (Dupont)	keep them under ~20 cm
2	USB Mini-B cables	for programming, then for standalone power
1-2	USB power bank or 5 V phone charger	to run the boards away from the PC
1	Common-cathode RGB LED + 3 × 330 Ω	optional master status indicator
1	Small breadboard	only needed for the RGB LED / pull-up resistors

3 Pin assignment

MCU pin	Function	Configuration	Morpho	Arduino	Used by
PB12	SPI2_NSS	master: GPIO out · slave: AF5	CN10-16	–	chip select, active low
PB13	SPI2_SCK	AF5	CN10-30	–	clock, master → slave
PB14	SPI2_MISO	AF5	CN10-28	–	slave → master (ACK)
PB15	SPI2_MOSI	AF5	CN10-26	–	master → slave (command)
PC13	B1 user button	input, pull-up	CN7-23	–	master only
PA5	LD2 user LED	output, push-pull	CN10-11	D13	both boards
GND	0 V reference	–	CN10-20	–	mandatory

Reading the headers

CN7 and CN10 are the two 2×19 “morpho” headers along the long edges. Pin 1 of each is the row nearest the ST-LINK part of the board, odd numbers are in the left column and even numbers in the right column when the board is seen from above with the USB connector upwards. Every pin name is printed on the underside of the PCB – when in doubt, identify a pin by its *name*, not by counting.

4 Circuit and wiring

Make every connection with the boards **unpowered**. The colours in the last column are the ones used in Figure 1 on the next page; any colour works electrically, but keeping to a convention makes a wiring fault obvious at a glance.

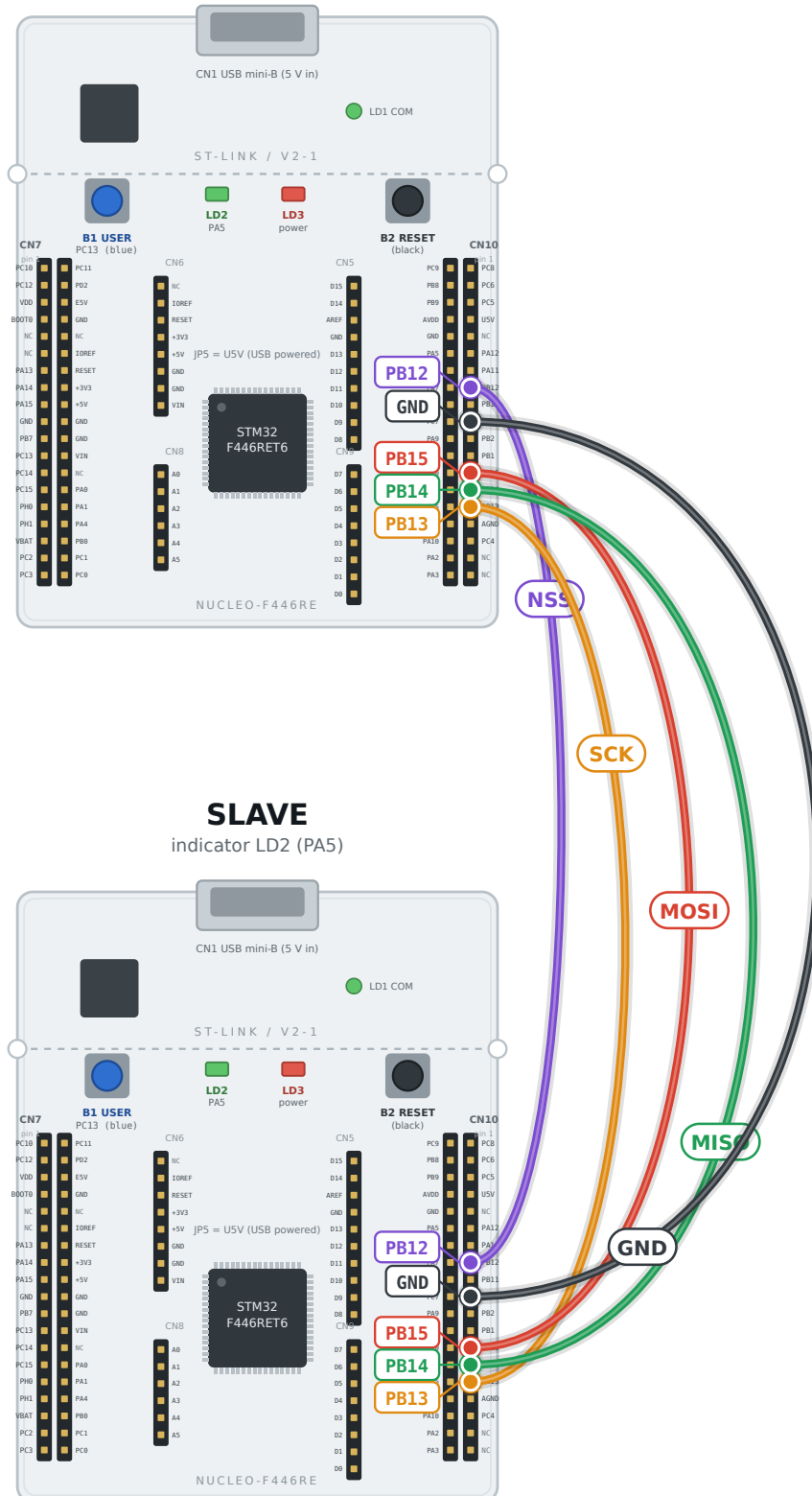
Signal	MASTER pin	SLAVE pin	Slave function	Wire colour used	Topology
NSS	PB12 · CN10-16	PB12 · CN10-16	NSS	violet	straight
SCK	PB13 · CN10-30	PB13 · CN10-30	SCK	orange	straight
MOSI	PB15 · CN10-26	PB15 · CN10-26	MOSI	red	straight
MISO	PB14 · CN10-28	PB14 · CN10-28	MISO	green	straight
GND	GND · CN10-20	GND · CN10-20	GND	black	common

Experiment 3 · SPI2 - board-to-board wiring

Top view, USB connectors upwards. Pin names are silk-screened on the underside of each board.

MASTER

button B1 (PC13) · status LED



SPI is wired STRAIGHT THROUGH: SCK-SCK, MOSI-MOSI, MISO-MISO. Do not cross MOSI and MISO.
PB12 = CN10-16, PB13 = CN10-30, PB14 = CN10-28, PB15 = CN10-26, GND = CN10-20.

Figure 1 - Physical wiring between the two boards, drawn to the real board outline. The coloured rings mark the pads that carry a wire.

5 Theory of operation

Why SPI2 and not SPI1?

`SPI1_SCK` shares pin PA5 with LD2, the green user LED that this experiment uses as its indicator. Choosing SPI2 (PB12–PB15 on the CN10 morpho header) keeps the LED free. Recognising this kind of pin conflict before wiring is half of interfacing work.

SPI is a **synchronous, full-duplex, single-master** bus built from two shift registers that form one circular 16-bit register. On every clock edge the master shifts one bit out on MOSI and one bit in on MISO; the slave does the mirror image. A byte can therefore never be sent without a byte simultaneously coming back – which is exactly how the acknowledgement is collected here for free.

Clock polarity, phase and speed

Both ends use **mode 0**: `CPOL = 0` (SCK idles low) and `CPHA = 0` (the data bit is sampled on the first, i.e. rising, edge and changed on the falling edge). The baud-rate field `BR = 101b` divides PCLK1 by 64, giving 16 MHz / 64 = **250 kHz**. That is deliberately slow: unshielded jumper wires have no controlled impedance, and 250 kHz has margin to spare. A slave has no BR field at all – it is clocked entirely by the master.

Chip select

The master drives NSS by hand as an ordinary GPIO (`SSM = 1`, `SSI = 1` internally so the peripheral never thinks it lost arbitration). The slave uses **hardware NSS**: `SSM = 0` and PB12 in AF5, so its shift register only moves while the master holds the line low. That is what keeps the two byte-counters aligned.

The four-byte transaction

Byte	MOSI (master → slave)	MISO (slave → master)	Purpose
1	0xAA	don't care	sync pattern
2	0xA1 / 0xA2	don't care	the command
3	~cmd	don't care	checksum
4	0x00 dummy	0x5A	clock the acknowledgement out

The slave writes `0x5A` into `SPI2->DR` as soon as the checksum byte validates, so it is already sitting in the transmit register when the master clocks the fourth byte. Only after that fourth byte has been received does the slave start blinking – otherwise the bus would be stalled for up to six seconds.

5.1 Clock configuration used by both boards

Both listings begin with `clock_init_hsi_16mhz()`, which selects the internal 16 MHz RC oscillator, leaves every bus prescaler at /1 and sets zero flash wait states. Nothing is left to a start-up file or to CubeMX, so **SYSCLK = HCLK = PCLK1 = PCLK2 = 16 MHz** and every timing number in this manual can be verified by hand. `SysTick` is then reloaded with 15999 to interrupt once per millisecond, which is the time base for both the button measurement and the blink pattern.

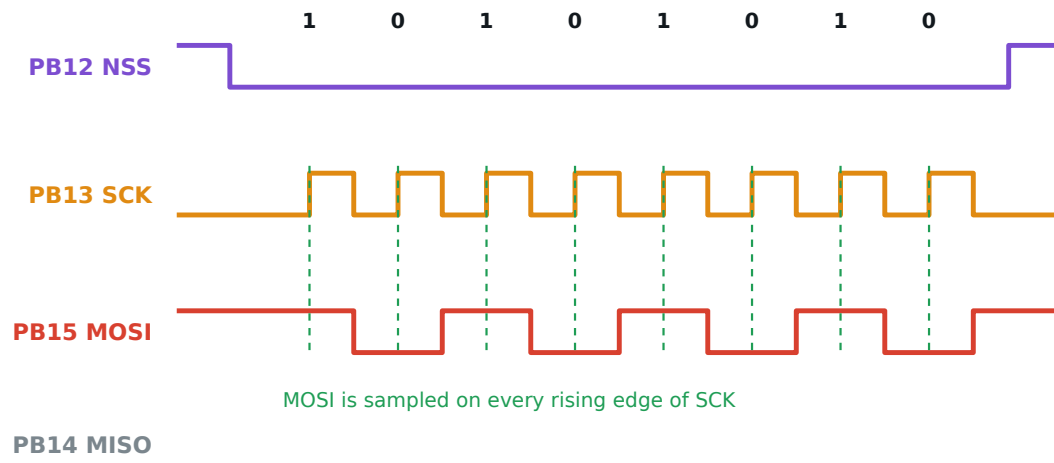
5.2 Register configuration summary

Register	Value written	Why
<code>RCC->APB1ENR</code>	<code> = SPI2EN</code>	clock to SPI2
<code>GPIOB->MODER</code>	PB13/14/15 = <code>10</code>	alternate function; on the master PB12 stays a plain output
<code>GPIOB->AFR[1]</code>	PB12–PB15 = <code>5</code>	AF5 = SPI2
<code>SPI2->CR1</code> (master)	<code>MSTR BR=101 SSM SSI SPE</code>	master at PCLK1/64 with software NSS

SPI2->CR1 (slave)	SPE only	MSTR = 0 and SSM = 0, so NSS comes from the PB12 pin
SPI2->CR2	0	no interrupts, no DMA, no hardware NSS output
SPI2->SR	poll TXE , RXNE , OVR	transmit register empty / byte received / byte lost
SPI2->DR	8-bit access	with DFF = 0 the register must be accessed a byte at a time, hence the <code>*(volatile uint8_t*)&SPI2->DR</code> cast

6 Signal timing

SPI2 mode 0 (CPOL = 0, CPHA = 0) - first byte of the 4-byte frame



0xAA leaves MSB first while the slave returns don't-care data; the ACK byte 0x5A arrives during the 4th byte.

Figure 2 – SPI mode 0. The master owns SCK, so there is no baud-rate agreement to get wrong; the slave simply shifts on the edges it is given.

7 MASTER source code `spi_master.c`

Paste this whole listing over the contents of `Core/Src/main.c` in the master project. It is self-contained: there are no other files to add and no header to edit.

`spi_master.c`

```
1/*=====
2 * EXPERIMENT 3 : SPI (SPI2, FULL DUPLEX)          --- M A S T E R ---
3 * Board : NUCLEO-F446RE (STM32F446RET6, Cortex-M4)
4 * Style : 100 % bare-metal CMSIS register programming - no HAL, no CubeMX.
5 *
6 * WHY SPI2 AND NOT SPI1
7 * SPI1_SCK is multiplexed with PA5, which is the on-board LD2 LED. Using
8 * SPI2 (PB12..PB15) keeps LD2 free for the visual indication.
9 *
10 * BUS SETTINGS
11 * Mode 0 : CPOL = 0 (clock idles low), CPHA = 0 (sample on 1st edge)
12 * MSB first, 8-bit frames, BR = 101b -> fPCLK1 / 64 = 250 kHz
13 * (deliberately slow so that ordinary jumper wires work reliably)
14 * NSS is driven by software from PB12 as a plain GPIO output.
15 *
16 * TRANSACTION (4 bytes, one chip-select frame)
17 * byte 0 0xAA sync          slave returns don't-care
18 * byte 1 cmd (0xA1 / 0xA2)  slave returns don't-care
19 * byte 2 ~cmd checksum      slave returns don't-care
20 * byte 3 0x00 dummy         slave returns ACK = 0x5A
21 *
22 * WIRING (straight through - SPI is not crossed like UART!)
23 * PB12 NSS (CN10-16) -> PB12 NSS (CN10-16)
24 * PB13 SCK (CN10-30) -> PB13 SCK (CN10-30)
25 * PB14 MISO (CN10-28) <- PB14 MISO (CN10-28)
26 * PB15 MOSI (CN10-26) -> PB15 MOSI (CN10-26)
27 * GND (CN10-20) <-> GND (CN10-20)
28 *
29 * Optional RGB status LED on the MASTER (common cathode + 3 x 330 R):
30 * PC0 = RED (CN7-38), PC1 = GREEN (CN7-36), PC2 = BLUE (CN7-35).
31 *=====*/
32
33#include "stm32f446xx.h"
34#include <stdint.h>
35
36/*----- user configuration -----*/
37#define USE_RGB          1u
38#define RGB_COMMON_CATHODE 1u
39
40#define LED_PIN          5u      /* PA5 - LD2 green user LED          */
41#define BTN_PIN          13u     /* PC13 - B1 blue user button        */
42#define NSS_PIN          12u     /* PB12 - software chip select        */
43
44#define CMD_SYNC          0xAAu
45#define CMD_SHORT         0xA1u
46#define CMD_LONG          0xA2u
47#define ACK_BYTE          0x5Au
48
49#define T_DEBOUNCE_MS     20u
50#define T_LONG_MS         1000u
51
52/*===== 1. CLOCK : HSI 16 MHz, no PLL =====*/
53static void clock_init_hsi_16mhz(void)
54{
55    RCC->CR |= RCC_CR_HSION;
56    while ((RCC->CR & RCC_CR_HSIRDY) == 0u) { }
57
58    FLASH->ACR = FLASH_ACR_LATENCY_0WS;
59
60    RCC->CFGR &= ~RCC_CFGR_SW;
61    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) { }
62
63    RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);
```

```

64}
65
66/*===== 2. SysTick 1 ms time base =====*/
67static volatile uint32_t s_ticks;
68
69void SysTick_Handler(void) { s_ticks++; }
70
71static void systick_init(void)
72{
73    SysTick->LOAD = 16000u - 1u;
74    SysTick->VAL = 0u;
75    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
76                  SysTick_CTRL_TICKINT_Msk |
77                  SysTick_CTRL_ENABLE_Msk;
78}
79static uint32_t millis(void) { return s_ticks; }
80static void delay_ms(uint32_t ms)
81{
82    uint32_t t0 = s_ticks;
83    while ((uint32_t)(s_ticks - t0) < ms) { __NOP(); }
84}
85/* ~40 us guard time so the polling slave can service each byte in time. */
86static void guard_delay(void)
87{
88    volatile uint32_t i;
89    for (i = 0u; i < 200u; i++) { __NOP(); }
90}
91
92/*===== 3. LD2 and B1 =====*/
93static void led_init(void)
94{
95    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
96    (void)RCC->AHB1ENR;
97
98    GPIOA->MODER &= ~(3u << (LED_PIN * 2u));
99    GPIOA->MODER |= (1u << (LED_PIN * 2u));
100    GPIOA->OTYPER &= ~(1u << LED_PIN);
101    GPIOA->OSPEEDR &= ~(3u << (LED_PIN * 2u));
102    GPIOA->PUPDR &= ~(3u << (LED_PIN * 2u));
103    GPIOA->BSRR = (1u << (LED_PIN + 16u));
104}
105static void led_on (void) { GPIOA->BSRR = (1u << LED_PIN); }
106static void led_off(void) { GPIOA->BSRR = (1u << (LED_PIN + 16u)); }
107
108static void button_init(void)
109{
110    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
111    (void)RCC->AHB1ENR;
112
113    GPIOC->MODER &= ~(3u << (BTN_PIN * 2u));
114    GPIOC->PUPDR &= ~(3u << (BTN_PIN * 2u));
115    GPIOC->PUPDR |= (1u << (BTN_PIN * 2u));
116}
117static uint32_t button_down(void)
118{
119    return ((GPIOC->IDR & (1u << BTN_PIN)) == 0u);
120}
121static uint8_t button_capture(uint32_t *press_ms)
122{
123    uint32_t t0, d;
124
125    if (!button_down()) return 0u;
126    delay_ms(T_DEBOUNCE_MS);
127    if (!button_down()) return 0u;
128
129    t0 = millis();
130    while (button_down()) { __NOP(); }
131    delay_ms(T_DEBOUNCE_MS);
132
133    d = millis() - t0;
134    if (press_ms != 0) { *press_ms = d; }
135    return (d >= T_LONG_MS) ? CMD_LONG : CMD_SHORT;

```

```

136}
137
138/*===== 4. Optional RGB indicator =====*/
139static void rgb_init(void)
140{
141    #if USE_RGB
142        uint32_t p;
143        RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
144        (void)RCC->AHB1ENR;
145        for (p = 0u; p <= 2u; p++) {
146            GPIOC->MODER &= ~(3u << (p * 2u));
147            GPIOC->MODER |= (1u << (p * 2u));
148            GPIOC->OTYPER &= ~(1u << p);
149            GPIOC->PUPDR &= ~(3u << (p * 2u));
150        }
151    #endif
152}
153static void rgb_set(uint32_t r, uint32_t g, uint32_t b)
154{
155    #if USE_RGB
156        #if RGB_COMMON_CATHODE
157            GPIOC->BSRR = (r ? (1u << 0) : (1u << 16)) |
158                (g ? (1u << 1) : (1u << 17)) |
159                (b ? (1u << 2) : (1u << 18));
160        #else
161            GPIOC->BSRR = (r ? (1u << 16) : (1u << 0)) |
162                (g ? (1u << 17) : (1u << 1)) |
163                (b ? (1u << 18) : (1u << 2));
164        #endif
165    #else
166        (void)r; (void)g; (void)b;
167    #endif
168}
169static void master_indicate(uint32_t ok)
170{
171    uint32_t i;
172    if (ok) {
173        rgb_set(0u, 1u, 0u);
174        led_on(); delay_ms(250u); led_off();
175    } else {
176        rgb_set(1u, 0u, 0u);
177        for (i = 0u; i < 3u; i++) { led_on(); delay_ms(60u); led_off(); delay_ms(60u); }
178    }
179    rgb_set(0u, 0u, 0u);
180}
181
182/*===== 5. SPI2 in MASTER mode =====*/
183static void spi2_master_init(void)
184{
185    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
186    RCC->APB1ENR |= RCC_APB1ENR_SPI2EN;
187    (void)RCC->APB1ENR;
188
189    /* PB12 = plain GPIO output, used as the chip-select line */
190    GPIOB->MODER &= ~(3u << (NSS_PIN * 2u));
191    GPIOB->MODER |= (1u << (NSS_PIN * 2u));
192    GPIOB->OTYPER &= ~(1u << NSS_PIN);
193    GPIOB->OSPEEDR |= (3u << (NSS_PIN * 2u));
194    GPIOB->PUPDR &= ~(3u << (NSS_PIN * 2u));
195    GPIOB->BSRR = (1u << NSS_PIN); /* CS idles HIGH */
196
197    /* PB13 = SCK, PB14 = MISO, PB15 = MOSI, all AF5 */
198    GPIOB->MODER &= ~(3u << (13u*2u)) | (3u << (14u*2u)) | (3u << (15u*2u));
199    GPIOB->MODER |= ((2u << (13u*2u)) | (2u << (14u*2u)) | (2u << (15u*2u)));
200    GPIOB->OTYPER &= ~(1u << 13u) | (1u << 14u) | (1u << 15u);
201    GPIOB->OSPEEDR |= ((3u << (13u*2u)) | (3u << (14u*2u)) | (3u << (15u*2u)));
202    GPIOB->PUPDR &= ~(3u << (13u*2u)) | (3u << (14u*2u)) | (3u << (15u*2u));
203    GPIOB->PUPDR |= (1u << (14u*2u)); /* pull-up on MISO */
204
205    GPIOB->AFR[1] &= ~(0xFu << ((13u-8u)*4u)) | (0xFu << ((14u-8u)*4u)) |
206        (0xFu << ((15u-8u)*4u));
207    GPIOB->AFR[1] |= ((5u << ((13u-8u)*4u)) | (5u << ((14u-8u)*4u)) |

```



```

208         (5u << ((15u-8u)*4u)));
209
210     SPI2->CR1 = 0u;
211     SPI2->CR2 = 0u;
212     SPI2->CR1 = SPI_CR1_MSTR      | /* master */
213         (5u << 3u)                | /* BR = 101 -> fPCLK1 / 64 = 250 kHz */
214         SPI_CR1_SSM                | /* software slave management */
215         SPI_CR1_SSI;              /* internal NSS high -> stay master */
216                                   /* CPOL = 0, CPHA = 0, MSB, 8-bit */
217     SPI2->CR1 |= SPI_CR1_SPE;
218 }
219 static void cs_low(void) { GPIOB->BSRR = (1u << (NSS_PIN + 16u)); }
220 static void cs_high(void) { GPIOB->BSRR = (1u << NSS_PIN); }
221
222 /* Full-duplex: writing a byte always reads one back. */
223 static uint8_t spi2_xfer(uint8_t out)
224 {
225     while ((SPI2->SR & SPI_SR_TXE) == 0u) { }
226     *(volatile uint8_t *)&SPI2->DR = out; /* 8-bit access to DR! */
227     while ((SPI2->SR & SPI_SR_RXNE) == 0u) { }
228     return *(volatile uint8_t *)&SPI2->DR;
229 }
230
231 /*===== main =====*/
232 int main(void)
233 {
234     uint8_t cmd, ack;
235     uint32_t press_ms, ok;
236
237     clock_init_hsi_16mhz();
238     systick_init();
239     led_init();
240     button_init();
241     rgb_init();
242     spi2_master_init();
243
244     rgb_set(0u, 0u, 1u); led_on(); delay_ms(300u); led_off(); rgb_set(0u,0u,0u);
245
246     for (;;) {
247         press_ms = 0u;
248         cmd = button_capture(&press_ms);
249         if (cmd == 0u) { continue; }
250
251         rgb_set(0u, 0u, 1u);
252
253         cs_low(); guard_delay();
254         (void)spi2_xfer(CMD_SYNC); guard_delay();
255         (void)spi2_xfer(cmd); guard_delay();
256         (void)spi2_xfer((uint8_t)(cmd ^ 0xFFu));
257         guard_delay();
258         ack = spi2_xfer(0x00u); /* the slave answers on this byte */
259         guard_delay();
260         cs_high();
261
262         rgb_set(0u, 0u, 0u);
263
264         ok = (ack == ACK_BYTE);
265         master_indicate(ok);
266     }
267 }

```

8 SLAVE source code `spi_slave.c`

Paste this whole listing over the contents of `Core/Src/main.c` in the slave project. It is self-contained: there are no other files to add and no header to edit.

`spi_slave.c`

```
1/*=====
2 * EXPERIMENT 3 : SPI (SPI2, FULL DUPLEX)          --- S L A V E ---
3 * Board : NUCLEO-F446RE (STM32F446RET6, Cortex-M4)
4 * Style : 100 % bare-metal CMSIS register programming - no HAL, no CubeMX.
5 *
6 * BUS SETTINGS (must be identical to the master except for MSTR)
7 *   MSTR = 0, CPOL = 0, CPHA = 0, MSB first, 8-bit frames.
8 *   NSS is HARDWARE managed here: SSM = 0 and PB12 is configured as AF5,
9 *   so the peripheral only shifts while the master holds PB12 low.
10 *   A slave has no baud-rate register - the master's SCK sets the speed.
11 *
12 * PROTOCOL
13 *   Receives [0xAA][cmd][~cmd] and pre-loads ACK (0x5A) into the transmit
14 *   register so that it is shifted out during the master's 4th (dummy)
15 *   byte. Only after the whole 4-byte frame has gone by does it drive the
16 *   blink pattern, so the SPI bus is never held up.
17 *
18 * WIRING (straight through)
19 *   PB12 NSS (CN10-16) <- master PB12
20 *   PB13 SCK (CN10-30) <- master PB13
21 *   PB14 MISO (CN10-28) -> master PB14
22 *   PB15 MOSI (CN10-26) <- master PB15
23 *   GND (CN10-20) <-> master GND
24 *
25 * BLINK SPECIFICATION
26 *   CMD_SHORT (0xA1) -> 3 blinks, 150 ms ON / 150 ms OFF (300 ms spacing)
27 *   CMD_LONG (0xA2) -> 3 blinks, 1000 ms ON / 1000 ms OFF (2 s spacing)
28 *=====*/
29
30#include "stm32f446xx.h"
31#include <stdint.h>
32
33#define LED_PIN          5u          /* PA5 - LD2 green user LED          */
34
35#define CMD_SYNC          0xAAu
36#define CMD_SHORT         0xA1u
37#define CMD_LONG          0xA2u
38#define ACK_BYTE          0x5Au
39
40#define SHORT_ON_MS       150u
41#define SHORT_OFF_MS      150u
42#define SHORT_REPEAT      3u
43#define LONG_ON_MS        1000u
44#define LONG_OFF_MS       1000u
45#define LONG_REPEAT       3u
46
47/*===== 1. CLOCK : HSI 16 MHz, no PLL =====*/
48static void clock_init_hsi_16mhz(void)
49{
50    RCC->CR |= RCC_CR_HSION;
51    while ((RCC->CR & RCC_CR_HSIRDY) == 0u) { }
52
53    FLASH->ACR = FLASH_ACR_LATENCY_0WS;
54
55    RCC->CFGR &= ~RCC_CFGR_SW;
56    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) { }
57
58    RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);
59}
60
61/*===== 2. SysTick 1 ms time base =====*/
62static volatile uint32_t s_ticks;
63
```

```

64void SysTick_Handler(void) { s_ticks++; }
65
66static void systick_init(void)
67{
68    SysTick->LOAD = 16000u - 1u;
69    SysTick->VAL = 0u;
70    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
71                    SysTick_CTRL_TICKINT_Msk |
72                    SysTick_CTRL_ENABLE_Msk;
73}
74static void delay_ms(uint32_t ms)
75{
76    uint32_t t0 = s_ticks;
77    while ((uint32_t)(s_ticks - t0) < ms) { __NOP(); }
78}
79
80/*===== 3. On-board LD2 =====*/
81static void led_init(void)
82{
83    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
84    (void)RCC->AHB1ENR;
85
86    GPIOA->MODER &= ~(3u << (LED_PIN * 2u));
87    GPIOA->MODER |= (1u << (LED_PIN * 2u));
88    GPIOA->OTYPER &= ~(1u << LED_PIN);
89    GPIOA->OSPEEDR &= ~(3u << (LED_PIN * 2u));
90    GPIOA->PUPDR &= ~(3u << (LED_PIN * 2u));
91    GPIOA->BSRR = (1u << (LED_PIN + 16u));
92}
93static void led_on (void) { GPIOA->BSRR = (1u << LED_PIN); }
94static void led_off(void) { GPIOA->BSRR = (1u << (LED_PIN + 16u)); }
95
96static void run_pattern(uint8_t cmd)
97{
98    uint32_t i, n, on, off;
99
100    if (cmd == CMD_LONG) { n = LONG_REPEAT; on = LONG_ON_MS; off = LONG_OFF_MS; }
101    else { n = SHORT_REPEAT; on = SHORT_ON_MS; off = SHORT_OFF_MS; }
102
103    for (i = 0u; i < n; i++) {
104        led_on(); delay_ms(on);
105        led_off(); delay_ms(off);
106    }
107}
108
109/*===== 4. SPI2 in SLAVE mode =====*/
110static void spi2_slave_init(void)
111{
112    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
113    RCC->APB1ENR |= RCC_APB1ENR_SPI2EN;
114    (void)RCC->APB1ENR;
115
116    /* PB12 = NSS, PB13 = SCK, PB14 = MISO, PB15 = MOSI - all AF5 */
117    GPIOB->MODER &= ~(3u << (12u*2u) | (3u << (13u*2u)) |
118                    (3u << (14u*2u)) | (3u << (15u*2u)));
119    GPIOB->MODER |= ((2u << (12u*2u)) | (2u << (13u*2u)) |
120                    (2u << (14u*2u)) | (2u << (15u*2u)));
121    GPIOB->OTYPER &= ~(1u << 12u | 1u << 13u | 1u << 14u | 1u << 15u);
122    GPIOB->OSPEEDR |= ((3u << (12u*2u)) | (3u << (13u*2u)) |
123                    (3u << (14u*2u)) | (3u << (15u*2u)));
124    GPIOB->PUPDR &= ~(3u << (12u*2u) | (3u << (13u*2u)) |
125                    (3u << (14u*2u)) | (3u << (15u*2u)));
126    GPIOB->PUPDR |= (1u << (12u*2u)); /* NSS pull-up: idle = not sel. */
127    GPIOB->PUPDR |= (2u << (13u*2u)); /* SCK pull-down: CPOL = 0 */
128
129    GPIOB->AFR[1] &= ~(0xFu << ((12u-8u)*4u) | (0xFu << ((13u-8u)*4u)) |
130                    (0xFu << ((14u-8u)*4u)) | (0xFu << ((15u-8u)*4u)));
131    GPIOB->AFR[1] |= ((5u << ((12u-8u)*4u)) | (5u << ((13u-8u)*4u)) |
132                    (5u << ((14u-8u)*4u)) | (5u << ((15u-8u)*4u)));
133
134    SPI2->CR1 = 0u; /* MSTR = 0, SSM = 0 -> hardware NSS */
135    SPI2->CR2 = 0u; /* CPOL = 0, CPHA = 0, MSB, 8-bit */

```

```

136     SPI2->CR1 |= SPI_CR1_SPE;
137}
138
139static uint32_t spi2_get(uint8_t *b)
140{
141     if (SPI2->SR & SPI_SR_OVR) { (void)SPI2->DR; (void)SPI2->SR; }
142     if (SPI2->SR & SPI_SR_RXNE) {
143         *b = *(volatile uint8_t *)&SPI2->DR;
144         return 1u;
145     }
146     return 0u;
147}
148static void spi2_load_reply(uint8_t b)
149{
150     if (SPI2->SR & SPI_SR_TXE) { *(volatile uint8_t *)&SPI2->DR = b; }
151}
152
153/*===== main =====*/
154int main(void)
155{
156     uint8_t byte, cmd = 0u, pending = 0u;
157     uint32_t state = 0u;
158
159     clock_init_hsi_16mhz();
160     systick_init();
161     led_init();
162     spi2_slave_init();
163
164     led_on(); delay_ms(300u); led_off();          /* "slave alive" signal */
165
166     for (;;) {
167         if (!spi2_get(&byte)) { continue; }
168
169         switch (state) {
170             case 0u:
171                 if (byte == CMD_SYNC) { state = 1u; }
172                 break;
173
174             case 1u:
175                 cmd = byte;
176                 state = 2u;
177                 break;
178
179             case 2u:                                /* checksum byte */
180                 if ((uint8_t)(cmd ^ 0xFFu) == byte &&
181                     (cmd == CMD_SHORT || cmd == CMD_LONG)) {
182                     spi2_load_reply(ACK_BYTE);      /* goes out on byte 4 */
183                     pending = cmd;
184                     state = 3u;
185                 } else {
186                     state = 0u;
187                 }
188                 break;
189
190             default:                                /* the dummy 4th byte */
191                 run_pattern(pending);               /* ACK has been shifted */
192                 pending = 0u;
193                 state = 0u;
194                 break;
195         }
196     }
197}

```

9 Building it in STM32CubeIDE without touching CubeMX

1. **File → New → STM32 Project.** In the target selector open the *Board Selector* tab, type `NUCLEO-F446RE` and select it (or use the *MCU/MPU Selector* and pick `STM32F446RET6`).
2. Give the project a name, keep *Targeted Language* = **C** and *Targeted Binary Type* = **Executable**, and – this is the important one – set **Targeted Project Type = Empty**. An *Empty* project gives you the startup assembly file, the linker script, `system_stm32f4xx.c` and the CMSIS register headers, but **no HAL, no LL drivers and no .ioc file**. There is therefore nothing to configure graphically – which is exactly what a bare-metal experiment requires.
3. Answer **No** if the IDE offers to initialise peripherals; with an *Empty* project that dialog normally does not appear at all.
4. Open `Core/Src/main.c`, delete its whole contents and paste the listing from this manual. The single include `#include "stm32f446xx.h"` resolves through `Drivers/CMSIS/Device/ST/STM32F4xx/Include`, which the Empty project already put on the include path.
5. The symbol `STM32F446xx` is predefined by the project wizard. If the compiler complains that the header cannot decide which device it is, add it manually under *Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor*.
6. Build with **Ctrl + B**. Connect the first board, then *Run → Run As → STM32 C/C++ Application* to flash it.
7. Repeat steps 1–6 in a second project for the other listing, and flash it to the second board. Label the boards **MASTER** and **SLAVE** with a marker – they look identical once they are off the PC.

Alternative without the IDE: copy the generated `.bin` onto the `NODE_F446RE` mass-storage drive that appears when the board is plugged in, or use STM32CubeProgrammer.

10 Running the pair with no PC attached

Nothing in these programs depends on the debugger. The moment the board is powered, the reset vector in flash runs `main()`, so the two boards work as a stand-alone pair.

Option A – one USB supply per board (recommended)

1. Unplug both boards from the PC after flashing.
2. Plug each board's CN1 Mini-B socket into a USB power bank, a 5 V phone charger or a powered USB hub. Leave jumper **JP5 on U5V** (its factory position).
3. LD3 (red) lights up on each board and both LD2s give a single ~300 ms flash: that is the power-up self-test built into the firmware.
4. Keep the protocol wires and the common ground connected exactly as during programming.

Option B – one 5 V source for both boards

1. Move **JP5 to E5V** on *both* boards.
2. Feed +5 V into `E5V` (CN7 pin 6) and 0 V into `GND` (CN7 pin 8 or CN10 pin 20) of both boards from the same supply.
3. The grounds are then already common, so the separate GND jumper of the protocol wiring becomes redundant – but leaving it in place does no harm.

Safety

Never drive 5 V into the `+5V` pin while JP5 is on U5V *and* a USB cable is plugged in – you would connect two supplies together. Choose one powering scheme and stay with it. BOOT0 must remain low (its default) so the MCU boots from flash.

11 Test procedure and observations

1. Flash the master listing to board 1 and the slave listing to board 2.
2. Power both boards down, then wire them exactly as in the connection table. Check the ground wire twice – it is the single most common cause of a dead link.
3. Power both boards. Each LD2 must give one short flash: both CPUs are alive.
4. **Test 1 – short click.** Press and release B1 on the master quickly (< 1 s). The master should flash green/LD2 and the slave should give three fast blinks.
5. **Test 2 – long press.** Hold B1 for at least one second, then release. The slave must now blink three times with one second on and one second off, i.e. two seconds between the start of one blink and the start of the next.
6. Time the slow pattern with a stopwatch and record it. Repeat each test three times.
7. **Test 3 – error case.** Pull the data wire out and press the button again. The master must show the red / triple-flash failure indication instead of green.
8. If an oscilloscope or logic analyser is available, capture the line(s) during one press and compare the capture with the timing figure in section 6.

Trial	Press duration (ms)	Expected command	Master indication	Blinks counted	Measured spacing (s)	Pass
1						
2						
3						
4						
5						
6						

12 Troubleshooting

Symptom	Cause and remedy
The slave receives nothing	MOSI and MISO are crossed. Unlike UART, SPI is wired straight through.
The master reads 0xFF or 0x00 instead of the ACK	MISO is not connected, or the slave was reset in the middle of the frame. Power the slave first, then the master.
Received bytes are shifted by one bit	CPOL/CPHA differ between the two boards, or NSS is not connected so the slave never re-aligns between frames.
Works at 250 kHz but fails when you raise BR	Expected with long unshielded jumpers. Shorten the wires before increasing the clock.
Neither LD2 flashes at power-up	The board is not running: check LD3 (red) is lit, that the USB supply really delivers power, and that you flashed the board you think you flashed.
The slave never reacts, master always shows red	Ground is not common. Join GND (CN10-20) of the two boards.
Behaviour is erratic, blinks appear at random	Wires too long or badly seated. Use jumpers under 20 cm and push them fully home.
Both boards blink but the patterns are swapped	The master listing was flashed to the slave board. Label the boards.
Long press is detected as a short one	You released before 1000 ms. The threshold is <code>T_LONG_MS</code> in the master listing – change it there if your lab asks for a different value.

13 Review questions

1. Why does SPI need no baud-rate agreement while UART does?
2. Explain CPOL and CPHA and give the four SPI modes.
3. What is the role of NSS, and what is the difference between SSM = 0 and SSM = 1?
4. Why can a byte never be received on SPI without one being sent?
5. Why was SPI2 chosen instead of SPI1 on this board?
6. How many devices can share one SPI bus, and what limits it?
7. What does the OVR flag mean on a slave, and how is it cleared?
8. Compare the pin count and the maximum speed of SPI, I2C and UART.